



SIGNAL SERIES • OBSERVABILITY ON KUBERNETES

Observability Study Guide

النسخة العربية المدمجة الكاملة — بناء منظومة Observability متكاملة على **Loki** Kubernetes: **Promtail** + للسجلات، **Prometheus + Exporters** للمقاييس، و**Grafana** للعرض — منشورة على **Minikube** عبر **Helm**. الأساسيات ← المعمارية ← النشر ← LogQL/PromQL ← ربط التطبيقات ← حل المشكلات ← النموذج الذهني ← المصطلحات ← الاستعداد للمقابلات.

By Nabawy

AUTHOR & COMPILER • إعداد وتنسيق

النسخة العربية

Arabic · Merged · A4

Loki

+ Promtail 📄

Prometheus

+ Exporters 📊

Grafana

Dashboards 📊

Minikube

+ Helm *

المحتويات

01	الخريطة — الملخص التنفيذي	3	08	Helm + kube-prometheus-stack — النشر 1/2	15
02	الركائز الأربع Observability ما هي ؟	4	08.b	loki-stack + النشر 2/2	16
03	Logs vs Metrics vs Traces — جدول المقارنة	5	09	LogQL — الاستعلام 1/2	17
03.b	The Core Stack — نظرة شاملة	6	09.b	PromQL — الاستعلام 2/2	18
03	Logs 1/3 — لماذا Loki؟	7	10	App Integration — ربط التطبيقات	19
03.b	Logs 2/3 — معمارية Loki	8	11	دفتر حل المشكلات 1/2	20
03.c	Logs 3/3 — سير عمل Promtail	9	11.b	حل المشكلات 2/2 + التنظيف	21
04	Metrics 1/2 — أساسيات Prometheus	10	13	النموذج الذهني — المعمارية	22
04.b	Metrics 2/2 — مصادر المقاييس	11	13.b	مراجعة سريعة — جدول المصطلحات	23
05	Traces — رحلة الـ Request	12	14	الاستعداد للمقابلات — 14 سؤالاً	24
06	Grafana — الواجهة الموحدة	13	14.b	Cheat Sheet — ورقة الأوامر	25
07	Kubernetes كيف تتجمع القطع في	14	14.c	اختبار الذات + الإجابات	26

- **TASK** [دليل ألوان الحالة] status legend
- TEAL — العناوين وهيكل السلسلة ■ INDIGO — المفاهيم الجوهرية KEY IDEA ■ GREEN — التأكيدات والإنذارات ■ RED — الأعراض والأعطال ■ AMBER — التحذيرات والملاحظات

NOTE كيف تستخدم الكتاب: 14 جزءًا مترابطًا تدمج دليل المراجعة ودليل النشر في رحلة واحدة — من تعريف الـ Observability، عبر أنواع الإشارات الثلاثة ومعماريات Loki و Prometheus، وصولاً إلى نشر Helm + Minikube الكامل، استعلامات LogQL/PromQL، ربط التطبيقات، دفتر حل المشكلات، جدول المصطلحات، والاستعداد للمقابلات. كل المصطلحات التقنية بالإنجليزية كما تُستخدم في بيئة العمل فعليًا، والشرح بالعربية.

KEY IDEA الـ Lab الذي يوثقه هذا الكتاب: Cluster واحد على Namespace Minikube · observability · Chart · ان من Helm (kube-prometheus-stack + loki-stack) ونموذج Labels واحد يلحم المنظومة كلها.

الملخص التنفيذي — الطريق كاملاً

قبل الغوص في أي أداة، هذه القصة كاملة — كل جملة رابط يأخذك إلى جزئها.

نبدأ في **الجزء 1 — ما هي Observability؟**: القدرة على فهم ما يحدث داخل النظام من البيانات التي ينتجها — بدون نشر Code جديد لطرح سؤال جديد — عبر أربع ركائز: **Logs** و **Metrics** و **Traces** و **Visualization**. ثم يضع **الجزء 2** أنواع الإشارات الثلاثة جنبًا إلى جنب — أهم جدول في الكتاب كله للامتحانات — ويعرّف **الجزء 2-b** بال Stack المنشور مكوّنًا مكوّنًا.

السجلات أولًا: **الجزء 3 — لماذا Loki؟** (أرخص 10-20 × — يفهرس ال Labels لا النص)، **الجزء 3-b** مسار الكتابة/القراءة (← Distributor Querier ← Store ← Ingester)، و**الجزء 3-c** سير عمل Promtail ك Ship ← Label ← Tail ← Discover ← DaemonSet.

المقاييس: **الجزء 4 — Prometheus** (نموذج السحب، ال PromQL، Time Series، اكتشاف ال Targets) و**الجزء 4-b** المصادر الأربعة: node-exporter و kube-state-metrics و cAdvisor و ServiceMonitor/PodMonitor CRDs. ثم **الجزء 5 — Traces** (الخطوة الطبيعية التالية) و**الجزء 6 — Grafana** الواجهة الموحدة عديمة الذاكرة.

كل ما هو عملي: **الجزء 7** تدفق البيانات الكامل • **الجزء 8** النشر بال Helm • **الجزء 8-b** فخ تعارض ال Datasource + ال Port-Forward • **الجزء 9** LogQL و **الجزء 10** ربط التطبيقات • **الجزء 11** دفتر حل المشكلات • **الجزء 13** النموذج الذهني • **جدول المصطلحات** • **الجزء 14** المقابلات وورقة الأوامر والاختبار. الطريق واضح. ▶

• TASK كيف تتعاون الإشارات [تدفق التصحيح الكلاسيكي] TASK

🔍 تنبيه Metric يُطلق "معدل الأخطاء فقر"

🔗 ال Trace يكشف ال Hop البطيء

📄 سطر Log يكشف ال Exception بالضبط

Grafana هو لوح الزجاج الذي يجمع كل هذا — ولأن Loki يستخدم نفس ال Label Model الخاص بـ Prometheus و Kubernetes تمامًا، يمكنك القفز من Metric على Pod إلى Logs ذلك ال Pod مباشرة.

KEY IDEA • الفكرة الجامعة كل مكوّن يقوم بعمل واحد بالضبط — Promtail يدفع ال Logs، و Prometheus يسحب ال Metrics،

و Grafana يستعلم الاثنين ويعرضهما. نموذجًا نقل مختلفان، ونموذج Labels مشترك واحد (namespace/pod/container/node). لا قواعد بيانات مشتركة، ولا تعقيد زائد.

What is Observability? — الركائز الأربع

Observability هي القدرة على فهم ما يحدث داخل النظام من خلال فحص البيانات التي يُنتجها — دون الحاجة إلى نشر Code جديد لطرح سؤال جديد. تعتمد على أربع ركائز أساسية:

TASK [الركائز الأربع] والسؤال الذي تجيبه كل ركن



Logs

"What happened, and what did it say?"

سجلات منفصلة مختومة بال Timestamp لكل حدث — الخطأ، التحذير، رسالة التشغيل.



Metrics

"How much, how fast, how often?"

قياسات رقمية عبر الزمن — CPU، ال Requests في الثانية، زمن الاستجابة.



Traces

"Where did the time go?"

المسار الكامل ل Request واحد وهو ينتقل عبر الخدمات المختلفة.



Visualization

"Can a human act on this?"

يحوّل الإشارات الخام إلى Dashboards يمكن للإنسان قراءتها والتصرف بناءً عليها.

الركائز الثلاث الأولى تنتج وتخزن الإشارات — والرابعة تجعلها قابلة للفهم والفعل.

KEY IDEA الركائز لا تكرر بعضها — كل واحدة تجيب نوعًا مختلفًا من الأسئلة: ال **Log** يخبرك ماذا حدث بالتفصيل، وال **Metric** يخبرك بكم وبأي سرعة، وال **Trace** يخبرك أين ذهب الوقت. التحقيق الكامل يستخدم الثلاثة معًا.

NOTE • لماذا التعريف مهم؟ لاحظ العبارة المفتاحية: «بدون نشر Code جديد لطرح سؤال جديد». تجهّز النظام مرة واحدة؛ وبعدها أي سؤال جديد («لماذا كان ال Latency عاليًا الساعة 2 فجراً؟»، «أي Pod رمى الخطأ؟») يُجاب بالاستعلام عن بيانات موجودة أصلاً — لا بإعادة نشر مع print statements إضافية. نظام المراقبة الجيد يجب على أسئلة لم تتوقعها وقت البناء — عكس ال Monitoring التقليدي الذي يجب فقط على أسئلة مكتوبة مسبقًا.

EXAM سؤال الافتتاح في أي Interview: «ما ال Observability وركائزها؟» ← فهم حالة النظام من بياناته دون نشر Code جديد • الركائز: **Logs • Metrics • Traces • Visualization** — مع السؤال الذي تجيبه كل ركن.

كيف تتحول الركائز إلى مكونات فعلية؟ ← **الجزء 2: Logs vs Metrics vs Traces**



Logs vs Metrics vs Traces

أنواع الإشارات الثلاثة متكاملة لا مكررة. هذا الجدول هو أهم قطعة في هذا الدليل للامتحانات والمقالات:

Logs 📄		Metrics 📊		Traces 🔗	
السؤال	ماذا حدث، وماذا قال؟	كم؟ بأي سرعة؟ كم مرة؟	أين ذهب الوقت؟		
الطبيعة	سجلات أحداث منفصلة مختومة بالوقت	قياسات رقمية عبر الزمن	رحلة Request واحد عبر الخدمات		
الشكل	أسطر نصية حرة	Time Series (اسم + Labels + قيم)	مسار ال Request عبر ال Hops		
مخزنٌ عندنا في	Loki	Prometheus	غير منشور في هذا ال Lab (كان سيُنشر (Tempo / Jaeger / Zipkin		
لغة الاستعلام	LogQL	PromQL	(واجهة ال Trace)		
الأفضل في	قراءة المحتوى الحرفي للحدث («أرني ال Stack Trace»)	الاتجاهات، المعدلات، التجميع، التنبيه، ال Dashboards	تفصيل ال Latency عبر Request متعدد الخدمات		

■ **TASK** [كيف تعمل معًا؟] تدفق التصحيح الكلاسيكي

تنبيه Metric يُطلق "معدل الأخطاء قفر"

Trace يكشف ال Hop البطيء

سطر Log يكشف ال Exception

Grafana هو لوح الزجاج الذي يجمع كل هذا معًا.

KEY IDEA لأن Loki يستخدم نفس ال Label Model الخاص ب Prometheus و Kubernetes تمامًا، تستطيع القفز من Metric على Pod إلى Logs ذلك ال Pod مباشرة. ترى قفزة CPU على `pod X, namespace Y` في Panel من Prometheus ← تستعلم `{namespace="Y", pod="X"}` في Loki ← تقرأ ال Logs من نفس النافذة الزمنية بالضبط. صفر إعدادات إضافية ينتج هذا.

EXAM 📌 Logs = ماذا حدث (Loki/LogQL) · Metrics = كم/بأي سرعة (Prometheus/PromQL) · Traces = أين ذهب الوقت (ليس في هذا ال Lab) · تدفق التصحيح: تنبيه ← Log ← Trace، وكلها ظاهرة في Grafana.

▶ الآن تعرّف على المكونات التي تخزن هذه الإشارات ← الجزء 2: The Core Stack

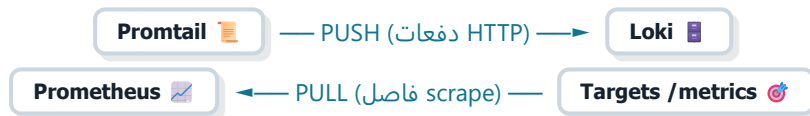


The Core Stack — نظرة شاملة

TASK [سبعة مكّونات، سبعة أدوار] من يفعل ماذا

المكوّن	الدور	يُنشأ بواسطة	يعمل كـ
Loki 📄	يخزّن ويفهرس الـ Logs	loki-stack	خلفية الـ Logs
Promtail 📄	يقرأ الـ Container Logs على كل Node ويشحنها إلى Loki	loki-stack	DaemonSet
Prometheus 📊	يسحب ويخزّن الـ Metrics كـ Time Series	kube-prometheus-stack	خلفية الـ Metrics
node-exporter 🖥️	يعرض مقاييس الـ Host	kube-prometheus-stack	DaemonSet
kube-state-metrics 📦	يعرض مقاييس حالة الـ K8s Objects	kube-prometheus-stack	Deployment
Alertmanager 📢	يوجّه شروط PromQL إلى Slack/Email/PagerDuty	kube-prometheus-stack	معطل في هذا الـ LAB
Grafana 📊	يستعلم الخلفيتين ويعرض Dashboards	kube-prometheus-stack	Deployment

TASK [واحد ناقلان، نموذج] الـ Labels



KEY IDEA الـ Labels المفتاحي الذي يجب تذكّره: Promtail يدفع الـ Logs إلى Loki؛ Prometheus يسحب الـ Metrics من الـ Targets. نموذجنا نقل مختلفان — ونموذج Labels مشترك واحد (namespace / pod / container / node) يجعل الـ Stack يبدو كنظام واحد.

KEY IDEA • قابلية التبديل هذه ثلاث مجموعات مستقلة تمامًا — يمكنك استبدال أي واحدة ببديل (ELK بدل Loki مثلاً) لأنها لا تتحدث مع بعضها مباشرة، بل تتقابل عند Grafana فقط. هذا الفصل هو سر مرونة الـ Stack.

WATCH OUT Grafana لا يخزّن شيئاً — عرض فقط. أي Dashboard بلا Data Sources حية = صفحة فارغة؛ البيانات تعيش دائماً في Loki أو Prometheus. ولاحظ أن Grafana نفسه منشور بواسطة kube-prometheus-stack لا مستقلاً — وهذه الحقيقة بالذات هي سبب فخ تعارض الـ Datasource في الجزء 8.b.

EXAM الـ Stack: **Loki + Promtail (Logs) · Prometheus + Exporters (Metrics) · Grafana (Visualization)** · Push = Logs، الـ Pull = Metrics · Promtail و node-exporter يعملان Alertmanager · DaemonSets معطل في الـ Lab · المكوّنات قابلة للتبديل — تتقابل عند Grafana فقط.

نبدأ بالركن الأول: لماذا Loki تحديداً؟ ← الجزء 3

Why Loki? — الفلسفة والفروق

Loki هو نظام تجميع Logs أفقي التوسّع (Horizontally-Scalable) يدعم Multi-Tenancy، مبني من **Grafana Labs**. يتميز عن أنظمة النص الكامل مثل ELK لأربعة أسباب جوهرية:

Cost-Effective — الأرخص 💰

فقط **Metadata** أرخص 10-20x في التخزين، لأنه يفهرس الـ **Labels** (فهرسة خفيفة = Log. لا النص الكامل لكل سطر — فاتورة تخزين خفيفة).

Storage — تخزين رخيص 🗄️

Object Storage وتُخزّن في **Chunks** تُضغط إلى Logs الـ **Search Nodes** بلا — (Labs محلي في الـ Filesystem أو) رخيص ضخمة.

Querying — LogQL 🔍

لغة مصممة عمدًا لتكون شبيهة بـ **LogQL** بـ **Loki** تستعلم **PromQL**. تعلّم واحدة والثانية تبدو مألوفة.

Integration — تكامل أصلي 🔗

ويستخدم نفس، (بلا واجهة منفصلة) **Grafana** تكامل أصلي مع **Prometheus** و **Kubernetes** الخاص بـ **Label Model** الـ بالضبط.

■ TASK [Loki vs ELK — الفرق في سطر واحد] المقايضة

ELK: يفهرس النص الكامل ← بحث نصي قوي 🔍

Loki: يفهرس الـ Labels فقط ← أرخص 10-20x 💰

KEY IDEA • النموذج الذهني + الفلسفة فُكّر في Loki كـ «**Prometheus للـ Logs**». والفلسفة في سطر: «لا تفهرس ما لا تستعلم عنه». فهرسة الـ Metadata (أي Namespace؟ أي Pod؟ أي Container؟) تكفي لتصفية ملايين الأسطر، وترك النص خامًا يُقرأ عند الاستعلام فقط — هذا هو الجذر المشترك لخصته وسلوك استعلاماته: محددات الـ Labels سريعة، وفلتر الأسطر تُدار لحظة الاستعلام.

WATCH OUT المقايضة حقيقية: لأن Loki لا يفهرس نص الـ Log نفسه، البحث النصي الحر أضعف من ELK. أنت تستعلم بالـ Labels أولًا ثم تضيّق بالـ LogQL. في بيئات Kubernetes ذات الـ Labels الجيدة — وهذه حالتنا بالضبط — المقايضة مجدية جدًا.

EXAM الأسباب الأربعة: **Cost** (أرخص 10-20x — فهرسة Labels فقط) • **Storage** (Chunks في Object Storage رخيص) • **LogQL** (شبيه بـ PromQL) • تكامل **Grafana** أصلي + نفس الـ **Label Model**. النموذج الذهني: «**Prometheus للـ Logs**» • الفلسفة: «لا تفهرس ما لا تستعلم عنه».

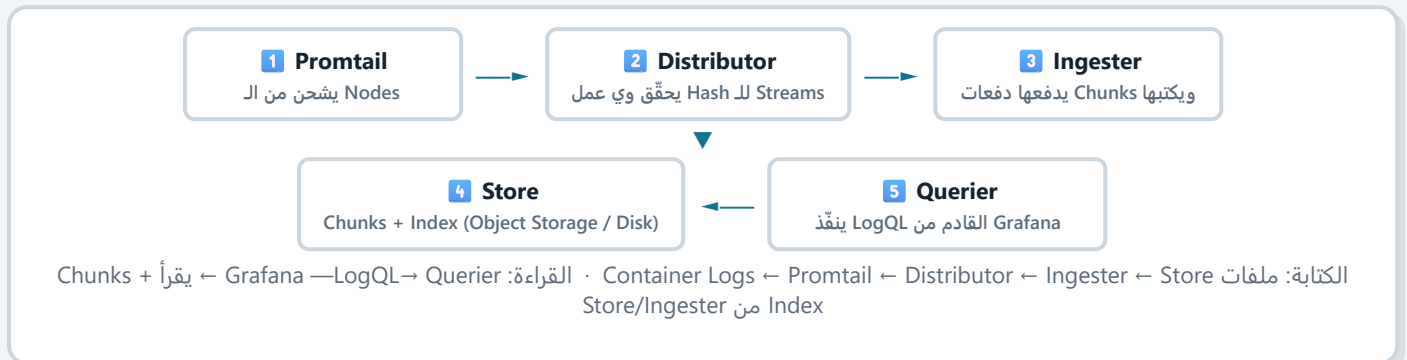
🔗 كيف يعمل Loki داخليًا؟ مسارا الكتابة والقراءة: الجزء 3-b



Loki Architecture — مسار الكتابة والقراءة

A-3 ال Pipeline بمراحله الخمس

Store والقراءة تعود من ال Pipeline عبر مسار محدد — الكتابة تنزل عبر ال Loki تعبر معمارية Logs ال



المكوّن	مهمته
1. Promtail	يشحن ال Logs من ال Nodes
2. Distributor	يحقّق ويعمل Hash لتدفقات ال Logs الواردة
3. Ingester	يدفع ال Streams دفعات Chunks ويكتبها
4. Store	يحفظ ال Chunks وال Index (Object Storage أو Disk محلي)
5. Querier	ينقذ استعلامات LogQL القادمة من Grafana

KEY IDEA ال Log Stream = مزيج Labels فريد. كل مجموعة Labels مختلفة هي Stream مستقلة بذاتها — ولهذا بالضبط تهم كثيرًا ال Labels التي يلحقها Promtail (الجزء 3-c).

NOTE · القراءة مقابل الكتابة الكتابة تتدفق يسارًا→ يمينًا عبر ال (4←1) Pipeline. القراءة لا تلمس مقدمة مسار الكتابة: ال Querier (5) يسحب ال Chunks + Index المخزنة من ال Store (ومن ال Ingestor للبيانات الحديثة غير المفرغة بعد).

EXAM ال Pipeline: Promtail ← Distributor (يحقّق ويعمل Hash) ← Store (Chunks + Index) ← Ingestor (Chunks) ال Querier (LogQL). ال Log Stream = مزيج Labels فريد.

المكوّن الذي يغذي المرحلة 1 — ولماذا هو DaemonSet: ال الجزء 3-Promtail c: >>

الخطوات الأربع + DaemonSet لماذا Promtail Workflow

يعمل Promtail كـ **Pod** — **DaemonSet** واحد لكل **Node** في الكلستر. لماذا؟ لأن Container Logs هي ملفات على الـ **Filesystem المحلي لكل Node**. و**Pod Promtail** على **Node A** يستحيل جسدًا أن يقرأ `/var/log/pods` الخاصة بالـ **Node B**. بيانات محلية للـ **Node** ← جامع محلي للـ **Node**.

■ TASK [الخطوات الأربع] discover ← tail ← label ← ship

1 Discover — اكتشف

لاكتشاف Kubernetes الخاص بـ **Service Discovery** يستخدم الخاصة به **Node** يعمل على الـ **Pod** كل.

2 Tail — اقرأ

مباشرة من `/var/log/pods` — Container Logs يقرأ ملفات الـ `tail -f` مثل أمر.

3 Label — علّم

Labels: **namespace, pod, container, node** كـ Kubernetes Metadata يلحق الـ

4 Ship — اشحن

عبر **Loki** الخاص بـ **Distributor** يدفع دفعات الأسطر إلى **HTTP**.

KEY IDEA • الـ **Labels** هي الترس في الماكينة الخطوة 3 أهم مما تبدو: لأن Labels الـ Promtail مطابقة تمامًا لـ **Labels** الـ

Prometheus، يصبح التنقل بين الـ Logs والـ Metrics في Grafana سلسيًا — نفس الـ **Namespace**، نفس الـ **Pod**، نفس اللغة — والـ Logs والـ Metrics قابلة للربط المتبادل افتراضيًا. ترى قفزة CPU على `pod X, namespace Y` في Panel من Prometheus ← تستعلم `{namespace="Y", pod="X"}` في Loki ← تقرأ الـ Logs من نفس النافذة. صفر إعدادات إضافية ينتج هذا.

WATCH OUT • Cardinality الـ **Labels** مفهومة إذًا مكلفة نسبيًا في Loki — لا تلصق قيمًا متغيرة بلا حدود (مثل `user_id`) كـ **Labels**. التزم بالثوابت المنظمة: **namespace, app, pod, container, node**.

NOTE الخطوة 4 تكمل نموذج الدفع: Promtail يدفع دفعات عبر HTTP — عكس Prometheus الذي يسحب من الـ Targets. نفس الـ **Labels**، واتجاه نقل معاكس.

EXAM 📌 Promtail = **DaemonSet** (Pod لكل Node — الـ Logs ملفات محلية في `/var/log/pods`) • الخطوات: **Tail** ← **Discover** **Label** ← (نفس الـ Labels الـ K8s كما في Prometheus) ← **Ship** (دفع HTTP) • الـ Labels المشتركة = ربط Logs↔Metrics مجاني.

من الـ Logs إلى الـ Metrics: ← **الجزء 4: Prometheus**



الأساسيات — Prometheus

Prometheus هو نظام مراقبة مفتوح المصدر يستخدم نموذج السحب (Pull Model): يسحب (Scrape) الـ Metrics من الـ Targets على فاصل زمني منتظم، بدلاً من انتظار التطبيقات لدفع البيانات إليه.

■ TASK [المفاهيم الثلاثة الجوهرية]

تخزين • استعلام • تنبيه

Storage 📁

اسم: **Time Series** كل قياس هو

Metric + Labels + سلسلة + مفتاح/قيمة

قيم مختومة بالوقت. مثال:

```
container_cpu_usage_seconds_total{namespace="api-7d9f"}
```

هي الأبعاد التي تجمع Labels — الـ عليها.

Querying — PromQL 🔍

لغة استعلام قوية للتجميع

والفلتر وحساب (Aggregation)

Time فوق بيانات الـ (Rates) المعدلات Series.

Alerting — Alertmanager 📢

لتحويل **Alertmanager** يتزامن مع

إلى تنبيهات موجهة PromQL شروط

Slack وEmail وPagerDuty. (معطل في) لكن المفهوم قائم — هذا الـ

■ TASK [لماذا يهم السحب؟]

نتائج عملية

سيطرة مركزية الـ Scraper هو من يقرر الإيقاع وقائمة الـ Targets — التطبيقات لا تحتاج لمعرفة مكان Prometheus أصلاً. فقط اكشف نقطة /metrics ، ولا شيء أكثر.

الفشل مرئي الـ Target الميت لا يترك فجوة في البيانات فقط — مقياس الـ up عنده يصبح 0 . تستطيع حرفياً الاستعلام عما إذا كانت المراقبة نفسها تعمل.

■ TASK [Targets اكتشاف الـ]

يدوية IPS بلا قوائم

في Kubernetes يجد Prometheus الـ Targets تلقائياً

via

Service Discovery + Custom Resources (CRDs)

لا أحد يصون قائمة عناوين Pods — الـ Pods زائلة أصلاً؛ القائمة ستكون قديمة خلال دقائق.

WATCH OUT كل مزيج Labels فريد يُنشئ **Series جديدة** — نفس مسألة انفجار الـ Labels في Loki: كثرة الـ Labels المتغيرة = انفجار في عدد الـ Series.

Pull Model Prometheus: الـ Time Series = اسم + Labels + قيم مختومة بالوقت • الاستعلام: **PromQL** • التنبيه:

Alertmanager (معطل هنا) • اكتشاف الـ Targets: تلقائي عبر **CRDs + Service Discovery** • الـ Target الميت = `up == 0`

من أين تأتي الـ Metrics فعلاً؟ ← **الجزء 4b: مصادر المقاييس**

Metric Sources — من يعرض ماذا

السؤال الذي يحبه	ماذا يعرض	أين يعمل	المكوّن
"هل الـ Machine نفسها سليمة؟"	مقاييس مستوى الـ Host : CPU و Memory و Disk و Network	DaemonSet (Pod لكل Node)	node-exporter 🖥️
"هل يظن Kubernetes أن كل شيء سليم؟"	حالة الـ Objects : حالة الـ Pods و Replicas و Restarts	يراقب Kubernetes API	kube-state-metrics 📦
"أي Container يلتهم الموارد؟"	استهلاك كل Container على حدة (الـ CPU/Memory الفعلي)	مدمج في الـ kubelet	cAdvisor ⚙️
"كيف يجد Prometheus تطبيقي؟"	قواعد تصريحية لأي Services/Pods يسحب Prometheus	CRDs — ليست Pods أصلاً	ServiceMonitor / PodMonitor 🎯

KEY IDEA • احفظ هذا التمييز: node-exporter = الـ **Objects** = kube-state-metrics = الـ **Machine** الـ cAdvisor = **Kubernetes** = الـ **Containers**. ثلاث طبقات من «هل هو سليم» — وثلاثة مكوّنات مختلفة. والـ ServiceMonitor CRD هو ما يوصل تطبيقك بـ Prometheus بلا أي Config يدوي.

TASK [من يراقب ماذا؟] الطبقات

node-exporter 🖥️ ← الـ Node نفسها

cAdvisor ⚙️ ← الـ Containers

kube-state-metrics 📦 ← حالة الـ K8s Objects

ServiceMonitor 🎯 ← أي Targets تُسحب

NOTE • لماذا الـ **CRDs** مهمة؟ بدل أن تسأل «ما الـ IPs التي أسحبها؟» — تنشئ ServiceMonitor يذكر الـ Service بالـ Label، و Helm Chart الذي ينشر التطبيق يُرفق ServiceMonitor معه. النتيجة: نشر التطبيق = مراقبته تلقائياً. نشر الـ CRD هو الإعداد. (مثال كامل في [الجزء 10](#).)

EXAM 📌 **node-exporter** (DaemonSet — مقاييس الـ Host) · **kube-state-metrics** (API Server — حالة الـ Objects) · **cAdvisor** (kubernetes — لكل Container) · **ServiceMonitor/CRDs** (تحديد السحب تصريحيًا، مربوطة تلقائياً بالـ Helm).

الركن الثالث: رحلة الـ Request الواحد ← [الجزء 5: Traces](#)

Traces — رحلة ال Request

ال Trace يسجل الرحلة الكاملة ل Request واحد وهو ينتقل عبر عدة Services:

■ TASK [وحدة Request رحلة] مثال محلول

2ms API Gateway



15ms Auth Service



40ms Order Service



△ 120ms DB Query

ال Trace يكشف فورًا: أغلب الوقت ذهب إلى ال DB Query — لا تخمين، ولا فحص يدوي لكل Service.

5.1 الغرض

Hop تخبرك أي Traces بطيئة؛ وال Requests تخبرك أن ال Metrics ال Request تُظهر بالضبط أين صُرف الوقت أثناء ال Traces ال تباطأت، أي استدعاء تكرر، وأين الاختناق الحقيقي Service جعلها بطيئة — أي

5.2 الأدوات

OpenTelemetry مع تجهيز التطبيقات عبر Zipkin أو Jaeger أو Grafana Tempo يستخدم عادةً Tracing ال

NOTE • في هذا ال Lab ال Traces غير منشورة في هذا ال Lab تحديدًا — لكنها تمثل الخطوة الطبيعية التالية بعد وجود ال Logs (Loki) وال Metrics (Prometheus). اثنتان من الإشارات الثلاث تعملان الآن.

KEY IDEA عندما يقول لك أحدهم «ال API بطيء» — ال Logs تخبرك ماذا حدث، وال Metrics تخبرك بكم هو بطيء، لكن ال Trace وحده يخبرك أين بالضبط ذهب الوقت داخل رحلة ال Request.

EXAM 🎓 ال Trace = رحلة Request واحد عبر Services متعددة · الغرض: أين صُرف الوقت · الأدوات: Grafana Tempo / Jaeger / Zipkin عبر OpenTelemetry · غير منشور في هذا ال Lab — إنه الخطوة التالية.

الركن الرابع: النافذة على كل شيء ← الجزء 6: Grafana

Grafana — الواجهة الموحدة

Grafana هو الواجهة الموحدة لمنظومة ال Observability — لوح الزجاج الذي يلتقي عنده ال Logs وال Metrics.

■ TASK [أربعة مفاهيم] Explore · Dashboards · البيانات · الإعداد · ال

التعامل مع البيانات 📊

Data Sources لا يخزن البيانات بنفسه — يتصل بـ Grafana ويستعلمها مباشرة عند كل تحديث للـ (Prometheus وLoki) ما يعرض — إنه Grafana امسح الخلفيتين وليس لدى Panels. عدسة لا قاعدة بيانات.

Configuration ⚙️

لغة + **Authentication** + **URL**: يحتاج Data Source كل **LogQL** أو **PromQL** الاستعلام التي سيتحدثها.

Dashboards & Panels 📊

يشغل Panel كل **Panels** مجموعة من ال **Dashboard** ال **Log Stream** أو **Table** أو **Graph** استعلامًا ويعرضه كـ **Gauge**. ال **Version-** قابلة للـ — **JSON** تُكتب Dashboards وال **Control** ID. أو الاستيراد من grafana.com/dashboards برقم

Explore Mode 🔍

دون بناء **LogQL** أو **PromQL** من **Ad-hoc** تشغيل استعلامات **Dashboard** — أرني أخطاء ذلك ال) الأداة المفضلة للتصحيح (Namespace الآن).

■ TASK [Grafana التسلسل الهرمي داخل] التسلسل

Dashboard 📊

تحتوي ◀

Panels ⚙️

تستعلم من ▶

Data Source 📊

وهو ▶

Loki / Prometheus 📊

KEY IDEA Grafana عديم الذاكرة عمدًا: كل **Panel** = **استعلام حي** في لحظة المشاهدة. لا يسأل أحد «هل ال Dashboard محدّث؟» — إنه محدّث بحكم التعريف، لأنه يستعلم المصدر مباشرة عند كل تحميل.

✓ استغل هذه الميزة · **Dashboards** كـ **Code** ال **Dashboards** ملفات **JSON** — **ضعها في Dashboard** **Git**. مفقود = استرجاعه من **Git**؛ وتغيير مقصود = **Pull Request** يُراجع مثل أي **Code** آخر. إدارة إصدارات ال **Dashboards** ميزة تُستغل، لا مصادفة.

WATCH OUT · **تفصيلة نشر** في هذا ال **Grafana**، **Stack** منشور بواسطة **kube-prometheus-stack** لا مستقلاً. هذه الحقيقة هي بالضبط سبب فخ تعارض ال **Datasource** في **الجزء 8-b** — أفضل قصة «عطل صلّحته» في هذا الدليل.

EXAM 📌 **Grafana**: لا يخزن — **Data Sources** حية · ال **Auth** + **URL** = **Data Source** + لغة استعلام · **Dashboard** = مجموعة Panels **JSON** ← **Dashboards** · (graph/table/logs/gauge): استرجاع + مراجعة Explore · **PR** = تصحيح **Ad-hoc**.

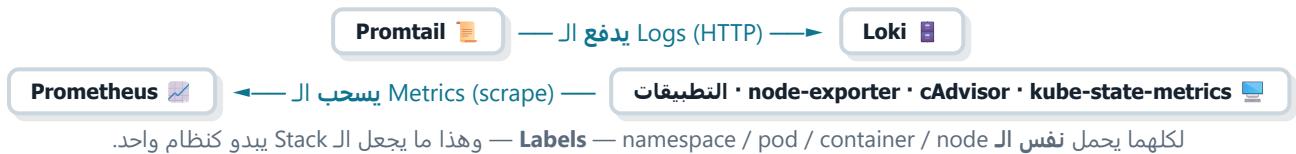
📺 كل القطع جاهزة — كيف تتجمع في Kubernetes ؟ ← **الجزء 7**

How It All Fits Together — في Kubernetes

TASK [تدفق البيانات الكامل] ست خطوات من طرف لطرف

1. **Pods التطبيقات** تكتب Logs وتكشف (اختياريًا) نقطة `/metrics`.
2. على كل **Promtail** (DaemonSet)، يقرأ `/var/log/pods`، يلحق كل سطر بـ Metadata الـ Kubernetes، ويدفع دفعات إلى Distributor الخاص بـ Loki.
3. على كل **node-exporter** (DaemonSet)، يعرض مقاييس الـ Host؛ و**cAdvisor** (داخل الـ kubelet) يعرض مقاييس كل Container؛ و**kube-state-metrics** يعرض حالة الـ Objects.
4. **Prometheus** يسحب كل تلك المقاييس على فاصل scrape، مكتشفًا الـ Targets عبر Service Discovery والـ CRDs (ServiceMonitor/PodMonitor).
5. **Grafana** يحمل الاثنين كـ Data Sources ويستعلم **Prometheus (PromQL)** و**Loki (LogQL)** مباشرة.
6. تفتح Grafana في المتصفح عبر نفق **port-forward** وترى كل شيء.

TASK [واحد ناقلان، نموذج] الـ Labels



KEY IDEA الـ Logs تُدفع؛ والـ Metrics تُسحب. اتجاهات النقل تختلف، لكن مفردات الـ Labels متطابقة — وهذا النموذج المشترك هو اللاصق. لهذا تكفيك قفزة Labels واحدة في Grafana لتنتقلك من Graph للـ CPU إلى أسطر الـ Logs التي تشرحه.

NOTE • نمط الوصول Grafana Service من نوع ClusterIP — يُوصل من داخل الكلستر فقط. من حاسبك تصل إليه عبر `kubect1 port-forward` (الأمر الكامل وتشرّحه في [الجزء 8-b](#)).

EXAM التدفق: **Apps/Nodes** تولّد ← **Promtail** يدفع **Prometheus** · **Loki** ← **Logs** يسحب **Grafana** · **TSDB** ← **Metrics** يستعلم الاثنين حيًّا ← المتصفح عبر `port-forward`. Push ≠ Pull، لكن الـ Labels مشتركة.

حان وقت النشر الفعلي — بأمرين من Helm: ← [الجزء 8: النشر](#)



Helm + kube-prometheus-stack

Helm هو مدير حزم لـ **Kubernetes**. يجمع كل ال Manifests اللازمة — Deployments و Services و ConfigMaps و CRDs — في «Charts». تثبيت Chart يُنشئ **Release**. وال Charts تعيش في **Repositories** تضيفها إلى جهازك. بدل عشرات ملفات YAML المكتوبة يدويًا، أمران من **Helm** ينشران المنظومة كاملة — بما فيها ال Dashboards الافتراضية وربط ال **Data Sources**. **Helm** حوّل مشروعًا من أسبوع إلى أمرين.

TASK [الإعداد] Repos + Namespace

مخصص Namespace : أفضل ممارسة

```
# أضاف المستودعين، حدّث الفهارس، أنشئ الـ Namespace
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
kubect1 create namespace observability # يُبقي المراقبة منفصلة عن أعباء التطبيقات
```

TASK [الـ Chart الأول: kube-prometheus-stack]

واحد، مكونات كثيرة Chart

يُثبت **ServiceMonitor/PodMonitor** ال **Prometheus** · **Alertmanager** · **Grafana** · **node-exporter** · **kube-state-metrics** **CRDs** افتراضية لـ **Kubernetes** — دفعة واحدة.

⚠ **إدارة الموارد** افتراضيًا يطلب هذا ال **Chart CPU** و **RAM** أكثر مما يستطيع **Cluster Minikube** صغير تحمّله — ثبتّه خامًا وستعلق ال Pods في **Pending** لأن ال Scheduler لا يجد مكانًا يتسع لطلبات الموارد. يجب تقليل **Resource Requests/Limits** لكل مكون **بشراصة** عبر أعلام **--set**:

```
helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
-n observability \
--set alertmanager.enabled=false \
--set prometheus.prometheusSpec.retention=6h \
--set grafana.adminPassword=admin123 \
--set prometheus.prometheusSpec.resources.requests.cpu=100m \
--set prometheus.prometheusSpec.resources.requests.memory=256Mi \
--set grafana.resources.requests.cpu=50m \
--set grafana.resources.requests.memory=128Mi
# kube-state-metrics و node-exporter. وما شابه من التقليل ...
# مفاتيح القيم الدقيقة: helm show values prometheus-community/kube-prometheus-stack
```

التعديل

لماذا

قيم ال Chart الافتراضية تفترض Nodes ضخمة؛ وإلا تعلق ال Pods في **Pending** على تقليل الموارد (لكل مكون) **Minikube**

تخزين ال TSDB يكبر مع مدة الاحتفاظ؛ ال Lab لا يحتاج تاريخًا — 6 ساعات تكفي للتصحيح **retention=6h**

لا حاجة لتوجيه تنبيهات في Pods ← Lab أقل RAM وأقل **alertmanager.enabled=false**

بيانات دخول Grafana في هذا ال Lab (admin123 / admin) **adminPassword=admin123**

🔑 قاعدة ال **IDEMPOTENCY** استخدم دائمًا **helm upgrade --install** وليس **helm install** المجرد. أمر

helm upgrade --install **helm install** **helm upgrade --install** آمن لإعادة التشغيل: إن كان ال Release موجودًا يرقّيه في مكانه؛ وإن لم يكن يثبتّه. أما **helm install** المجرد يفشل في التشغيل الثانية.

📌 **EXAM** **Helm** = مدير حزم **K8s** (Charts من Repos) · Chart واحد يثبت منظومة ال **Metrics** كاملة · **Minikube** يحتاج تقليل ال **Requests** وإلا **Pending** · **retention 6h** يوفر القرص · **helm upgrade --install** = Idempotent

📄 ال Chart الثاني — وأشهر فخ غير بديهي: **الجزء 8:b**

! loki-stack + الفخ الشهير + Port-Forward

```
helm upgrade --install loki-stack grafana/loki-stack \
  -n observability \
  --set grafana.sidecar.datasources.defaultDataSourceEnabled=false # العلم الحاسم
```

■ TASK [Datasource تعارض تزويد الـ] الفخ الوحيد غير البديهي

1. loki-stack يُرفق ConfigMap خاصًا به للـ Grafana Data-Source الخاص بـ Loki.
2. الـ Grafana Sidecar (العامل داخل Grafana المنشور من kube-prometheus-stack) يكتشف تلقائيًا ConfigMaps الـ Datasources ويحملها — فيظهر Loki في Grafana بلا أي إعداد يدوي. لطيف.
3. التعارض: kube-prometheus-stack يعيّن Prometheus كـ Datasource الافتراضي (Default). و loki-stack يحاول أيضًا تعيين Loki كافتراضي.
4. Grafana يسمح بافتراضي واحد بالضبط. اثنان افتراضيان = «Grafana» ← «Datasource provisioning error» يهدم — يعيد التشغيل ← يهدم ← CrashLoopBackOff.

✓ الحل ثبت مع Loki

```
--set
grafana.sidecar.datasources.defaultDataSourceEnabled=false
```

— يبقى Loki Datasource، لكن ليس الافتراضي (ويبقى Prometheus هو الافتراضي).

💡 إصلاح طارئ انكسر فعليًا؟

```
kubectl get configmap -n observability
```

الـ ConfigMap الزائد ← احذف السطر isDefault: true ← اعثر على
أعد تشغيل Pod الـ Grafana.

■ TASK [Port-Forward — Grafana الوصول إلى] ClusterIP = داخل الكلستر فقط

```
kubectl port-forward --address 0.0.0.0 -n observability \
  svc/kube-prometheus-stack-grafana 3000:80
```

admin / admin123 دخول ← http://localhost:3000 ثم افتح

المعنى	الجزء
الـ Service الذي سنحفر النفق إليه	svc/kube-prometheus-stack-grafana
الـ Namespace الذي يعيش فيه الـ Service	-n observability
البورت المحلي 3000 ← بورت الـ Service 80	3000:80
استمع على كل واجهات الجهاز لا الـ loopback فقط — إلزامي على Windows/WSL، وإلا تحصل على connection refused من جهة الـ Windows	--address 0.0.0.0

DASHBOARDS مثبتة مسبقًا (من kube-prometheus-stack): موارد الكلستر الحاسوبية، الـ Namespaces، الـ Nodes، والـ Kubelet — كلها Prometheus-backed. غير مشمولة: الـ Dashboards الـ Loki — استوردها من الواجهة (Load ← ID ← Import ← Dashboards):
(Log volume) 12611 · (Kubernetes Logs) 13639 — أو تجاوز الـ Dashboards واستخدم Explore.

📌 **EXAM** Chart ان يزودان Datasource افتراضيًا ← Grafana يهدم دوريًا (يسمح بواحد بالضبط) ← الحل:
defaultDataSourceEnabled=false · ClusterIP ← Grafana · port-forward مع --address 0.0.0.0 ← على WSL
admin/admin123 · استيراد 13639 / 12611 Dashboards.

🔗 الـ Stack واقف — الآن استعلم عنه: ← الجزء 9: LogQL

Loki الاستعلام عن LogQL

التأطير: خلفيتان، لغتا استعلام، واجهة واحدة. LogQL صُمم عمدًا ليكون شبيهًا بـ PromQL، وكلاهما يختار بنفس الـ Labels — تعلم واحدة والثانية تصبح مألوفة.

TASK [Explore ابدأ من وضع] Dashboard

Explore ← Grafana ← اختر Data Source ← شغل استعلامات Ad-hoc

Loki: {namespace="observability"} ← Streams حية

Prometheus: up ← 1 سليم، 0 ساقط

TASK [LogQL — مراحل ثلاث حتى] محدد ← فلترة ← غلاف رقمي

1 محدد الـ Stream

مفهرسة وسريعة. يختار أي — Labels الـ
ستقرأ Log Streams

2 Pipeline الفلترة

فلترة الأسطر — تُقَيِّم لحظة الاستعلام
المطابقة Streams على الـ

3 غلاف الـ Metric

Logs يحوّل عدّ الأسطر إلى أرقام — الـ
تصبح Metric.

```
# 1) Namespace الـ Logs كل Stream محدد الـ
{namespace="observability"}

# (Promtail التي لحقها Labels الـ) Pod / Container تضيق بالـ
{namespace="default", pod="my-app-7d9f8b6c-", container="app"}

# 2) (Loki هنا النص) فلترة الأسطر: احتفظ / احذف حسب المحتوى
{namespace="observability"} |= "error"
{namespace="observability"} |= "error" != "timeout"

# 3) مفي نافذة 5 Pod عدّ الأسطر المطابقة لكل Metric استعلام
sum by (pod) (count_over_time({namespace="default"} |= "error" [5m]))
```

KEY IDEA • لماذا يهم هذا الترتيب؟ محدد الـ Labels يُشغّل على الـ Index؛ والبحث النصي يُشغّل على الـ Streams المطابقة لحظة الاستعلام. هذا هو العائد المباشر من تصميم Loki «فهرس الـ Labels فقط» — التخزين الرخيص لا يعني استعلامات بطيئة إذا اخترت Labels جيدة.

NOTE • الجسر المرحلة 3 تحوّل الـ Logs إلى Metric (count_over_time) فوق نطاق [5m] — شقيق بنيوي لـ rate() في PromQL، والجسر بين نوعي الإشارات.

EXAM LogQL = محدد Stream (مفهرس) ← فلترة أسطر (لحظة الاستعلام) ← غلاف Metric اختياري • |= احتفظ، != احذف •
count_over_time(...[5m]) يحوّل أسطر الأخطاء إلى معدل لكل Pod.

النصف الآخر من الثنائي: الجزء 9: PromQL



Prometheus الاستعلام عن — PromQL

TASK [أمثلة جوهريّة — موسومة بالمكوّن المصدر] Metric اعرف من يعرض كل

```
# في السحب (1 = حي، 0 = ساقط) Target صحة كل
up
# نفسها فقط observability منظومة الـ
up{namespace="observability"}
# kubelet عبر) cAdvisor من Metric - m عبر 5 Container لكل CPU معدل
sum by (pod) (rate(container_cpu_usage_seconds_total[5m]))
# cAdvisor - الآن Container الـ Memory
sum by (pod) (container_memory_usage_bytes)
# node-exporter من Metric - ككسر Node المتاحة في الـ Memory
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes
```

TASK [آليتان تستحقان الاستيعاب] Range Vectors والتجميع

Range Vector = [5m]

يختار نقاطًا كثيرة خلال 5 دقائق — دوال مثل `rate()` تتطلب نطاقًا. قيمة لحظية واحدة لا تُنتج معدلًا.

sum by (pod) = تجميع

مع إبقاء `pod` ك بعد التجميع — نفس الـ Labels يجمع عبر الـ فتصطف النتائج عبر الخلفيتين، Loki التي يستخدمها Labels.

TASK [تصحيح لحظي أم اتجاهي؟] اختر السؤال الصحيح

«هل هو معطل الآن؟» ← Stream · up خام {namespace="..."}

«هل يتجه للتعطّل؟» ← rate() على [5m] · عدد الأخطاء/Graph · Pod كسر الـ Memory

KEY IDEA لأن لغتي الاستعلام تختاران نفس الـ Labels، يعمل نفس التجميع `sum by (pod)` على الجانبين — معدل الأخطاء لكل Pod من Loki يتطابق تمامًا مع معدل الـ CPU لكل Pod من Prometheus في Panels متجاورة داخل Grafana.

EXAM 🚩 = صحة السحب · `rate(counter[5m])` يحتاج Range Vector · `sum by (pod)` يُبقي label التجميع · cAdvisor ← مقاييس الـ Containers · node-exporter ← مقاييس `node_*`

إدخال تطبيقك أنت إلى الصورة: ← الجزء 10: ربط التطبيقات



ربط التطبيقات — Application Integration

TASK [ال Logs — صفر إعدادات] تعمل فوراً

Logs **Promtail** يجمع **Logs** كل **Pod** في الكلستر تلقائياً — بلا Annotations ولا CRDs ولا شيء. ال DaemonSet + Service ال Discovery يلتقطان ال Pods الجديدة فور ظهورها. انشر تطبيقاً، وLogs الخاصة به (موءوسة أصلاً بـ namespace/pod/container) تصبح في Loki.

TASK [ال Metrics — شرطان] اختيارية بطبيعتهما

1. على التطبيق أن يكشف نقطة `/metrics` على بورت `http` مسمى (توجد Client Libraries لكل لغة رئيسية تنشر Counters/Gauges/Histograms بصيغة Prometheus).
2. أنشئ `ServiceMonitor` **Custom Resource** يطابق ال Label Selector تطبيقك — فيكتشفه Prometheus عبر ال CRD (وHelm/Operator يوصل الباقي تلقائياً).

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app
  namespace: observability
  labels:
    release: kube-prometheus-stack # Prometheus Operator يجب أن يطابق ما يختاره ال
spec:
  selector:
    matchLabels:
      app: my-app # Labels تطبيقك أنت عبر Service يختار
  namespaceSelector:
    matchNames:
      - default # التطبيق Service حيث يعيش
  endpoints:
    - port: http # /metrics الذي يخدم Service بورت ال (NAME) اسم
      path: /metrics
      interval: 30s
```

selector.matchLabels

التطبيق **Service** يجد

endpoints.port

لهذا — Service يشير إلى اسم بورت ال `http` قلنا «على بورت

release ال Label

Operator ما تطابقه قواعد اكتشاف ال الذي «Auto-wiring هذا هو» ال — Helm يقدمه.

تحقق من النجاح استعلم `up{namespace="default"}` في Explore — تطبيقك يجب أن يظهر بقيمة 1 خلال فاصل scrape أو اثنين.

EXAM Logs = صفر إعدادات (Promtail يجمع تلقائياً) Metrics = نقطة / Label · **ServiceMonitor CRD** + **metrics** ال `release:` يربط ال CRD بال Operator · التحقق: `up{namespace="default"} == 1`.

▶▶ حين تسوء الأمور — الدفتر: ← الجزء 11: حل المشكلات

Troubleshooting — السيناريوهات 3-1

فكر بمنطق: عرض ← تشخيص ← سبب جذري ← إصلاح. تسلسل أوامر التشخيص أهم من حفظ الإصلاحات.

السيناريو 1 — Pods عالقة في Pending

العرض	kubectl get pods -n observability ← Pods جائمة في Pending لا تبدأ
التشخيص	kubectl describe pod <pod> -n observability ← الأحداث تُظهر FailedScheduling: Insufficient cpu / memory
السبب الجذري	الكليستر يفتقر CPU/Memory حرة — طلبات الموارد الافتراضية لل Chart لا تتسع على Node الـ Minikube، وال Scheduler لن يضع IPod لا تتسع طلباته
الإصلاح	(قلل الطلبات) (أعلام --set من الجزء 8) أو كَبِّر Minikube (--cpus=... --memory=...)
الدرس	Pending = مشكلة جدولة = انظر إلى الطلبات مقابل المتاح قبل أي شيء آخر

السيناريو 2 — Grafana بحالة OOMKilled

العرض	Grafana يعيد التشغيل باستمرار؛ kubectl describe pod يُظهر OOMKilled (Exit Code 137)
السبب الجذري	ال Container نفدت ذاكرته — تجاوز حد الـ Memory الخاص به
شخص أولاً	افحص ذاكرة الـ Node الحرة أولاً (kubectl top nodes) — لا ترفع حد Grafana بدم بارد
الإصلاح	ال Node فيها مجال ← ارفع الحد عبر --set ؛ الـ Node مكتظة ← حزّر الموارد (عطل مكونات، قلم Pods، كَبِّر Minikube)
الدرس	OOMKilled = مشكلة حد؛ إنهاك الـ Node = مشكلة سعة. شخص أي الاثنين أمامك.

السيناريو 3 — Grafana في CrashLoopBackOff مع خطأ Provisioning

العرض	Grafana يدور في CrashLoopBackOff ؛ Logs تُظهر «Datasource provisioning error»
التشخيص	kubectl logs <grafana-pod> -n observability · kubectl get configmap -n observability ← ستجد Default في ConfigMap، كلاهما Default
السبب الجذري	Datasource ان معلّمان Grafana — Default يسمح بواحد بالضبط؛ التزويد يفشل عند كل إقلاع ← حلقة هدم. ليست مشكلة موارد — فشل تحقق Config قد يُسقط IPod مضبوط الحجم تمامًا.
الإصلاح	وقاية: ثبت loki-stack مع defaultDatasourceEnabled=false · علاج: احذف isDefault: true من الـ ConfigMap الزائد ثم kubectl delete pod (الـ Deployment يعيد إنشاءه نظيفًا)
الدرس	اقرأ الانهيّار — رسالة الخطأ تسمّي النظام الفرعي (Provisioning)، وهذا يشير مباشرة إلى الـ ConfigMaps. هذه قصة المقابلة النجمية.

السيناريوهان 4-5 + التنظيف

السيناريو 4 — «Connection refused» من Windows / WSL

العرض	متصفح Windows لا يصل إلى localhost:3000 رغم أن ال port-forward يعمل
السبب الجذري	النق ارتبط على ال loopback داخل ال WSL VM فقط — كان علم --address 0.0.0.0 مفقودًا
الإصلاح	أعد الأمر كاملاً kubect1 port-forward --address 0.0.0.0 -n observability svc/kube-prometheus-stack-grafana 3000:80
الدرس	على WSL، متصفحك والكلستر تسكنهما حدود شبكية مختلفة — اربط كل الواجهات ليعمل الترحيل

السيناريو 5 — «Grafana لا يُظهر بيانات» (تسلسل الاستدلال العام)

اعمل على السلسلة بالترتيب — الأسباب المملة هي الأكثر شيوعًا:

- هل ال Pod الخلفية يعمل؟ ← `kubect1 get pods -n observability` (Loki أو Prometheus منهار = لا استعلام سينجح).
- هل ال Datasource حي؟ ← Explore ← شغل `up` (Prometheus) أو `{namespace="observability"}` (Loki). إن عمل Explore ولم يعمل Dashboard، فالمشكلة في استعلام/نطاق زمن ال Dashboard — لا في ال Stack.
- Metrics التطبيق مفقودة تحديدًا؟ ← لا نقطة `/metrics`، أو ال ServiceMonitor Selector لا يطابق — افحص `up` لل Target.
- ال Namespace الصحيح؟ النطاق الزمني الصحيح؟ ← الأسباب البسيطة تستحق نظرة أولًا.

12 التنظيف — إزالة ال Stack بالكامل

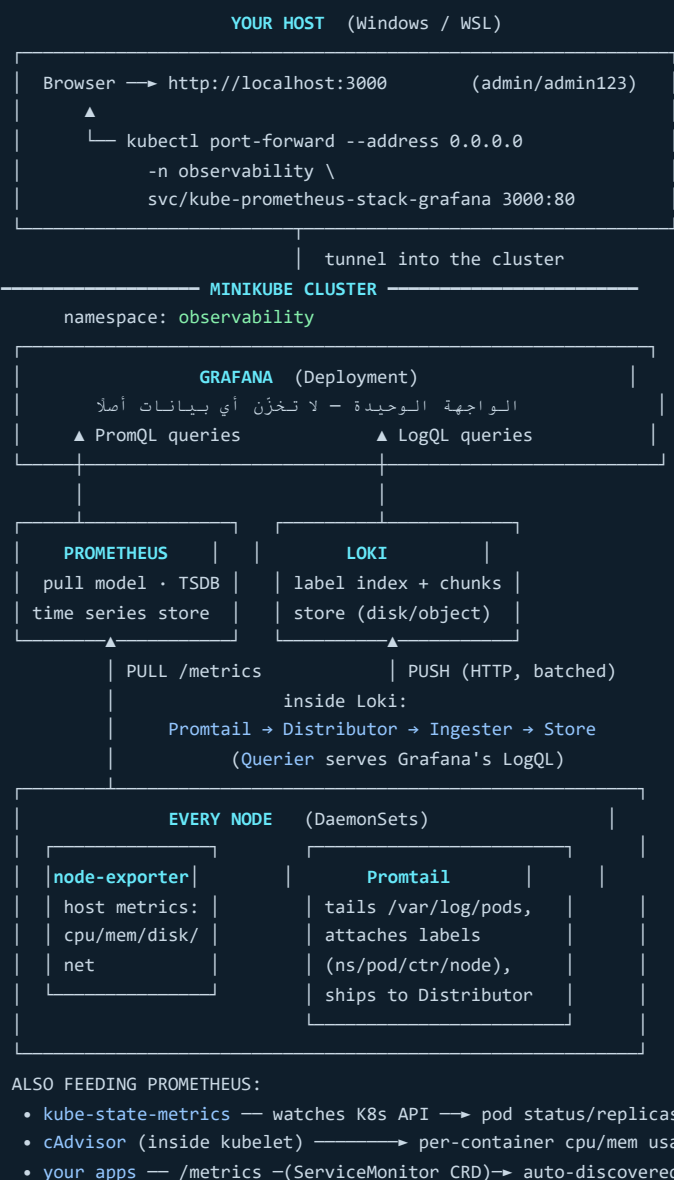
```
helm uninstall kube-prometheus-stack -n observability
helm uninstall loki-stack -n observability
kubect1 delete namespace observability
```

⚠ ملاحظة عملية حذف ال Namespace يزيل ما تركه ال Chart ان بدخله — لكن ال CRDs التي تبتها kube-prometheus-stack هي Cluster-Scoped وقد تبقى بعد ال Uninstall. احذفها يدويًا إن أردت Cluster نظيفًا تمامًا.

🚩 EXAM port-forward على WSL يحتاج --address 0.0.0.0 · سلسلة «لا بيانات»: ال Pod الخلفية ← ال Datasource (Explore) ← ال Target التطبيق ← ال Namespace/النطاق الزمني · التنظيف: helm uninstall مرتين + حذف ال Namespace (+ ال Cluster- CRDs Scoped يدويًا).

🔗 الآن اقرأ النظام كله من بعيد: الجزء 13: النموذج الذهني

The Mental Model — المعمارية في صفحة



🔴 النموذج الذهني في فقرة الـ Nodes تنتج إشارات خام. **Promtail** يدفع الـ **Logs** إلى Loki الذي يفهرس الـ **Labels** فقط ويضغط الـ Chunks في التخزين. **Prometheus** يسحب الـ **Metrics** من node-exporter و cAdvisor و kube-state-metrics وأي تطبيق يكشف `/metrics` + ServiceMonitor. كلا النظامين يتحدثان نفس لغة الـ **Grafana**. **Labels** (namespace/pod/container/node) لا يخزن شيئاً — يستعلم الخلفيتين جيّاً (PromQL/LogQL) ويعرضهما للبشر. وتصل إلى Grafana من حاسبك عبر **port-forward** لأن الـ ClusterIP الخاص به غير قابل للوصول خارجيّاً.



Quick Revision — جدول المصطلحات

كل مصطلح في هذا الكتاب، سطر واحد لكل مصطلح. إن استطعت تعريفها كلها دون نظر — فأنت جاهز.

TASK [Logs الإشارات + ال]

المصطلح	التعريف في سطر
Observability	فهم حالة النظام من بياناته هو — بدون نشر Code جديد لسؤال جديد
Log	سجل حدث منفصل مختوم بالوقت — «ماذا حدث وماذا قال»
Metric	قياس رقمي عبر الزمن — «كم، بأي سرعة، كم مرة»
Trace	رحلة Request واحد عبر الخدمات — «أين ذهب الوقت»
Visualization	Dashboards تحول الإشارات الخام إلى ما يمكن للإنسان الفعل بناءً عليه
Loki	خلفية تجميع Logs؛ يفهرس ال Labels فقط ويخزن Chunks مضغوطة
Promtail	DaemonSet يقرأ <code>/var/log/pods</code> على كل Node؛ يدفع إلى Loki
Distributor	يحقّق ويعمل Hash لتدفقات ال Logs الواردة
Ingestor	يدفع ال Streams دفعات Chunks ويكتبها
Store	يحفظ ال Chunks + ال Index (Object Storage أو Disk)
Querier	ينقذ استعلامات LogQL القادمة من Grafana
Log Stream	مزيج Labels فريد — كل مجموعة Labels مختلفة Stream = مستقلة
LogQL	لغة استعلام Loki: محدد Stream ← فلترة أسطر ← غلاف Metric
Chunk	دفعة أسطر Logs مضغوطة في Object Storage / Disk

TASK [Metrics العرض • النشر •]

المصطلح	التعريف في سطر
Prometheus	خلفية Metrics قائمة على السحب تخزن Time Series
Pull Model	Prometheus يسحب ال Targets على فاصل؛ ال Target الميت = <code>up 0</code>
Time Series	اسم Metric + Labels مفتاح/قيمة + قيم مختومة بالوقت
PromQL	لغة استعلام Prometheus — تجميع، فلترة، معدلات
Alertmanager	يوجّه شروط PromQL إلى Slack/Email/PagerDuty (معطّل هنا)
node-exporter	DaemonSet يعرض مقاييس ال Host — طبقة ال Machine
kube-state-metrics	يراقب ال K8s API — حالة ال Replicas Objects و Restarts
cAdvisor	مدمج في ال CPU/Memory — kubelet لكل Container
ServiceMonitor	CRD يصرّح بأي Services/Pods يسحب Prometheus
Grafana	الواجهة الموحدة؛ لا يخزن شيئاً — يستعلم ال Data Sources حيّاً
Data Source	URL + Auth + لغة استعلام (PromQL أو LogQL) داخل Grafana
Dashboard / Panel	ال Dashboard مجموعة Panels؛ كل Panel يعرض استعلاماً واحداً
Helm	مدير حزم Releases ← Charts من K8s: Repositories
DaemonSet	Pod واحد لكل Node — لل Collectors المحلية لل Node
port-forward	نفق من ClusterIP Service إلى localhost (WSL على <code>--address 0.0.0.0</code>)
Pending • OOMKilled • CrashLoopBackOff	حالات الفشل الثلاث: جدولة / حدّ الذاكرة / انهيار الإقلاع

طريقة المراجعة غيّ العمود الأيمن بيدك، اقرأ كل مصطلح، وقلّ التعريف بصوت مسموع. أي مصطلح تتعثّر فيه ← اقفز إلى جزئه: المصطلحات تطابق ال [فهرس المحتويات](#) واحداً لواحد.

أسئلة المفاهيم — بإجابات مركزة

Q1 ما هي الـ Observability؟

فهم ما يحدث داخل النظام من البيانات التي ينتجها — بدون نشر Code جديد لسؤال جديد. الركائز: Logs و Metrics و Traces و Visualization.

Q2 Logs vs Metrics vs Traces — متى تستخدم كلًا؟

Logs = ماذا حدث (أحداث حرفية). Metrics = كم وبأي سرعة (اتجاهات وتنبيه). Traces = أين ذهب الوقت (لكل Hop). تدفق التصحيح: تنبيه ← Trace ← Log.

Q3 لماذا Loki أرخص 10-20x من أنظمة النص الكامل؟

يفهرس الـ Metadata (Labels) فقط لا نص الـ Logs؛ النص يُبحث لحظة الاستعلام. والـ Chunks تُضغط في Object Storage رخيص.

Q4 Pull vs Push — نموذج Prometheus مقابل Promtail؟

Prometheus يسحب (scrape على فاصل — سيطرة مركزية على الإيقاع؛ والـ Target الميت مرئي عبر up=0). و Promtail يدفع دفعات HTTP إلى Distributor الخاص بـ Loki.

Q5 لماذا Promtail و node-exporter يعملان DaemonSets؟

بياناتهما محلية للـ Node: الـ Container Logs في /var/log/pods لكل Node؛ ومقاييس الـ Host لكل Pod لكل Machine. Node = تغطية كاملة.

Q6 مِم يتكوّن الـ Time Series في Prometheus؟

اسم Metric + Labels مفتاح/قيمة + سلسلة قيم مختومة بالوقت.

Q7 node-exporter vs kube-state-metrics vs cAdvisor؟

مقاييس الـ Machine (DaemonSet) مقابل حالة الـ Objects من API Server (حالة Pods و Replicas و Restarts) مقابل استهلاك كل Container المدمج في kubelet.

Q8 ما الـ ServiceMonitor/PodMonitor ولماذا وُجدتا؟

CRDs تخبر Prometheus تصرّحًا أي Services/Pods يسحب — بلا قوائم IPs يدوية؛ نشر الـ CRD هو الإعداد (Helm يوصله تلقائيًا).

Q10 لماذا helm upgrade --install لا helm install؟

الـ Idempotency — إعادة التشغيل ترقّي Release موجودًا في مكانه بدل الفشل.

Q11 لماذا port-forward لا NodePort/LoadBalancer؟

Grafana منشور ClusterIP (داخلي فقط). الـ port-forward = وصول Lab بلا إعدادات؛ NodePort/LB/Ingress هو ما تكشف به الخدمة رسميًا. على Windows/WSL يجب تمرير --address 0.0.0.0.

Q12 لماذا تُنزل الـ retention في Prometheus إلى 6h في Lab؟

تخزين الـ TSDB يكبر مع الـ retention؛ الـ Lab لا يحتاج تاريخًا — 6h توفر قرصًا معتبرًا.

Q13 كيف ترتبط الـ Logs بالـ Metrics في هذا الـ Stack؟

نفس الـ Promtail — Label Model يلحق الـ Labels نفسها التي يستخدمها Prometheus (namespace/pod/container/node)، فتترجم شذوذة Metric مباشرة إلى Stream الـ Logs لذلك الـ Pod.

Q14 ما الخطوة الطبيعية التالية لهذا الـ Stack؟

الـ Traces — انشر Grafana Tempo (أو Jaeger/Zipkin) وجّهز التطبيقات بـ OpenTelemetry — لتكتمل الإشارات الثلاث.

Q9 ★ احكِ لي عن Bug صلّحته في هذا الـ Stack.

Grafana في CrashLoopBackOff مع خطأ Datasource provisioning: كلا الـ Chart ينزّل Datasource معلنًا Default؛ Grafana يسمح بواحد بالضبط. شخّصته عبر الـ Logs الـ Pod + kubect1 get configmap وأصلحته بـ grafana.sidecar.datasources.defaultDatasourceEnabled=false (أو حذف isDefault: true).

📌 نصيحة المقابلة السؤال Q9 هو القصة النجمية — يُظهر تصحيحًا حقيقيًا: اقرأ Log الانهيار، كوّن فرضية من النظام الفرعي المسمّى في الخطأ (Provisioning)، تحقّق بالـ ConfigMaps، طبق الإصلاح الموجّه، واذكر الدرس (فشل تحقق Config قد يُسقط IPod مضبوط الحجم تمامًا).

ورقة الأوامر — Command Cheat Sheet

الأمر	ماذا يفعل / يخبرك
<code>helm repo add prometheus-community https://prometheus-community.github.io/helm-charts</code>	إضافة مستودع Charts
<code>helm repo add grafana https://grafana.github.io/helm-charts</code>	إضافة مستودع Charts
<code>helm repo update</code>	تحديث فهارس المستودعات
<code>kubectl create namespace observability</code>	Namespace مخصص للمنظومة
<code>helm upgrade --install <release> <chart> -n observability --set ...</code>	تنصيب/ترقية Idempotent
<code>helm list -n observability</code>	ال Releases المثبتة
<code>helm uninstall <release> -n observability</code>	إزالة Release
<code>kubectl get pods -n observability</code>	أول فحص صحة: Running / Pending / CrashLoopBackOff
<code>kubectl describe pod <pod> -n observability</code>	الأحداث: FailedScheduling و OOMKilled وسبب ال Restart
<code>kubectl logs <pod> -n observability</code>	أخطاء التطبيق (خطأ ال Provisioning يسكن هنا)
<code>kubectl get configmap -n observability</code>	إيجاد ConfigMaps تزويد ال Datasources
<code>kubectl top pods -n observability</code> / <code>kubectl top nodes</code>	ال CPU/Memory الفعلي مقابل ال Requests
<code>kubectl delete pod <pod> -n observability</code>	فرض إعادة تشغيل نظيفة (بعد إصلاح ConfigMap مثلاً)
<code>kubectl port-forward --address 0.0.0.0 -n observability svc/kube-prometheus-stack-grafana 3000:80</code>	الوصول إلى Grafana على localhost:3000

■ TASK [استعلامات السطر الواحد] احفظ هذه الأربعة

```
{namespace="observability"}                                     # Loki - Streams حبة من ال Namespace
up                                                            # سحب ناجح، 0 ساقط 1 - Prometheus
sum by (pod) (rate(container_cpu_usage_seconds_total[5m]))    # CPU لكل Pod (cAdvisor)
sum by (pod) (count_over_time({namespace="default"} |= "error" [5m])) # ال Pod الأخطاء لكل (LogQL)
```

تذكر ترتيب التشخيص الذي لا يفشل أبدًا: `get pods` ← `describe pod` ← `logs` ← `get configmap` ← `top nodes/pods`. كل سيناريو في الجزء 11 يستخدم مجموعة جزئية من هذه الخمسة بالضبط.

اختبار الذات + مفتاح الإجابات

1 أي مكوّن في Loki يحقّق ويعمل Hash لل Streams الواردة؟

2 من أي مسار يقرأ ال Promtail ال Container Logs، وبأي نوع ؟Workload

3 ما الأشياء الثلاثة التي تتكوّن منها ال Time Series؟

4 أي مكوّن يعرض CPU/Memory لكل Container، وأين يعمل؟

5 لماذا يهدم Grafana دوريًا عند وجود Datasource افتراضيين، وما علم `--set` الدقيق للإصلاح؟

6 ماذا يعني `up == 0` ؟

7 لماذا لا يخزّن Grafana أي بيانات بنفسه؟

8 ما الأوامر الثلاثة للتنظيف الكامل؟

9 ما ID ال Dashboard المجتمعية التي تغطيان ال Logs ال Kubernetes وحجم ال Logs؟

10 لماذا يجب أن يستخدم ال port-forward علم `--address 0.0.0.0` على Windows/WSL؟

■ TASK [مفتاح الإجابات] قس نفسك بصدق

1. ال **Distributor** (ثم ال Ingester يدفع ال Streams دفعات Chunks، وال Store يحفظ Chunks + Index).
2. `/var/log/pods` على ال Node الخاصة به؛ ال Promtail يعمل Pod — **DaemonSet** لكل ال Node، لأن ال Logs ملفات محلية لل Node.
3. اسم **Metric + Labels** مفتاح/قيمة + سلسلة قيم مختومة بالوقت.
4. **cAdvisor**، المدمج في ال **kubelet**.
5. Grafana يسمح بافتراضي واحد بالضبط؛ والتزويد يفشل عند كل إقلاع. الحل: `--set grafana.sidecar.datasources.defaultDatasourceEnabled=false` عند تثبيت loki-stack.
6. ال Target ساقط — لم يستطع Prometheus سحبه (الفشل المرئي مكسب من نموذج السحب).
7. إنه طبقة عرض: يتصل بـ Data Sources (Prometheus وLoki) ويستعلمها حيًا.
8. `helm uninstall kube-prometheus-stack -n observability` ← `helm uninstall loki-stack -n observability` ← `kubectl delete namespace observability`.
9. **13639** (Kubernetes Logs) و**12611** (Log volume)، تُستورد من واجهة Grafana.
10. بدون ال يرتبط النفق بال loopback داخل WSL فقط، فيحصل متصفح Windows على **connection refused**؛ و `0.0.0.0` يربط كل واجهات الجهاز.

الفحص النهائي إن استطعت رسم معمارية الجزء 13 من الذاكرة، وتعريف كل مصطلح في الجزء 13-b دون نظر، والإجابة على أسئلة المقابلات ال 14 بصوت مسموع، وتنفيذ النشر من ورقة الأوامر — فأنت تملك هذا ال **Stack**. انتهى الطريق هنا. ▶